

Home assignment #4: Gates and physical synthesis**Deadline:** Friday, December 22, 2023.

Summary: In this assignment you are required to fully synthesize your RISC-V processor from home assignment #1 with two open-source PDK libraries, NCSU FreePDK45 and SkyWater SKY130. You may also use another RISC-V implementation, but you must mention your source. The synthesis process consists of SRAM memory compilation with openRAM, gates synthesis with Synopsys Design Compiler, Physical synthesis (automatic place and route) with Cadence Innovus, and functional simulation with parasitic (SDF) back-annotation (gate-level simulation). Finally, you are required to design a run-in-batch synthesis manager that supports parameter sweep.

Details:

PART 1: Synthesis with FreePDK45 library (<https://eda.ncsu.edu/freepdk/freepdk45/>)

1.1. SRAM Memory compiler

Use the open-source SRAM compiler <https://openram.org/> to create SRAM blocks for all memories in your RISC-V implementation (such as instructions and data memory). If the generated file format is “.lib”, convert it to “.db” format using Synopsys library compiler (see slide set #9 - p. 58, or the tutorial <https://personal.utdallas.edu/~xxx110230/lc/>)

1.2. Gate synthesis with Synopsys Design Compiler: Based on slide set #9 and #10: (1) Specify libraries (link_library, target_library, symbol_library, and synthetic_library) to include the target PDK, Synopsys DesignWare, and SRAM. (2) Read design. (3) Set design environment. (4) Set design constraints. (5) Compile design. (6) Analyze design. (6) Generate netlist and timing files (SDC and SDF).

- Identify and address all errors and warnings during design reading, compilation, and analysis. Resolve all errors and critical warnings. If there are warnings that you consider non-critical and optional to fix, provide an explanation for your decision.
- Implement all arithmetic units (particularly those in the ALU) using Synopsys DesignWare.
- Identify any false paths and multi-cycle paths in the design and define them.
- Report the 5 most critical paths, explain them, and suggest a way to increase the timing margins on these paths.
- Try the suggested method to increase timing margins, recompile the design, and compare the deference.
- Approximately, find the maximum frequency (you may need to compile the design several times). Report the area and power for the maximum frequency.

1.3. Repeat the simulation in home assignment #2, but now with gates parasitic (SDF) back-annotation (gate-level simulation). See slide set #10, p. 70.

1.4. Perform physical synthesis (automatic placement and routing) using Cadence Innovus (See slide set #12 & #13). To invoke Innovus, see slide set #12, p. 33.

- Import your design (slide set #12, p. 41)
- Connect global nets (slide set #12, p. 47)

- Specify floorplan (slide set #12, p. 48)
- Add power rings (slide set #13, p. 6) and power stripes (slide set #13, p. 12)
- Perform placement (slide set #13, p. 12)
- Perform pre-CTS timing analysis (slide set #13, p. 19) and optimization (slide set #13, p. 20)
- Perform clock tree synthesis (CTS) (slide set #13, p. 21, 22, 28, 29)
- Perform post-CTS timing analysis (slide set #13, p. 19)
- Perform NanoRoute (slide set #13, p. 32)
- Perform post-route timing analysis (slide set #13, p. 19): explain the most critical path. Approximately, find the maximum frequency (you may need to recompile the design).
- Output data (slide set #13, p. 40, 41, 43)
- Report area and power.

1.5. Repeat the simulation in home assignment #2, but now with the post-routing parasitic (SDF) back-annotation (gate-level simulation). See slide set #9, p. 99, and slide set #10, p. 70.

PART 2: Synthesis with SkyWater SKY130 library

Redo STEP 1 using the SkyWater SKY130 library (<https://github.com/google/skywater-pdk>)

PART 3: Run-in-batch synthesis manager with parameter sweep support

Write a script (c-shell, bash, python, etc.) that synthesizes a design several times with different parameter values as indicated by the user and generates a CVS (comma-separated) table listing the area, power, and maximum frequency for each parameter value. For this step you may assume that the design is the parametric double dabble binary to BCD converter from:

<https://github.com/AmeerAbdelhadi/Binary-to-BCD-Converter/blob/master/bin2bcd.v>

Name your script “synManager”. The script should receive a parameter list as an argument. For instance, running your script: “./synManager 7 9 13 16 22” will synthesize 5 converters with 7-bit, 9-bit, 13-bit, 16-bit, and 22-bit. It will also generate the CSV table:

Param,	Fmax,	Area,	Power
7,	420,	20,	0.2
9,	405,	22,	0.3
13,	380,	27,	0.5
16,	370,	31,	0.8
22,	355,	36,	1.2

The table will be stored in the file “results.csv”. The values in the table above are just examples.

Important: Compose your report in a tutorial format, ensuring that you explicitly mention all the commands and tools you utilize, and include relevant screenshots.

Submission: By email, submit all documents, slides, and source code in a compressed file. Use the following file name convention: <ECE748C01F23-HA04-YourLastName>.zip

CAD tools instructions:

Using CMC Cloud Design Environments:

- (1) Read the instruction in <https://www.cmc.ca/qsg-cmc-cloud/>
- (2) Create a New Instance of "My Linux CAD Workstation (Centos 7)".
- (3) Use the NoMachine Client to connect.

Performing gates synthesis with Synopsys Design Compiler:

- (1) Source the setup script:
``source /CMC/scripts/synopsys.syn.2022.12.csh``
- (2) Run ``dc_shell`` to launch Design Compiler (DC).
- (3) Run ``dc_shell -gui`` or ``design_vision`` to launch DC graphical user interface. You may also run ``gui_start`` in `dc_shell` commandline prompt to launch the graphical user interface.
- (4) Run ``dc_shell -f script.tcl`` to launch DC and executing ``script.tcl``. You may also run ``source script.tcl`` in `dc_shell` commandline prompt to execute ``script.tcl``.

Performing physical synthesis (automatic placement & routing) with Cadence Innovus:

- (1) Source the setup script:
``source /CMC/scripts/cadence.innovus21.17.000.csh``
- (2) Run ``innovus`` to launch Cadence Innovus.

Performing simulation with Synopsys Verdi/VCS:

- (3) Source the setup script:
``source /CMC/scripts/synopsys.verdi.2021.09-SP1-1.csh``
- (4) Execute ``verdi`` to invoke Synopsys Verdi. Other executables include ``nWave``, a waveform viewer, and ``vcs``, a command-line simulator.

Performing simulation with Cadence Xcelium:

- (1) Source the setup script:
``source /CMC/scripts/cadence.xceliummain21.09.003.csh``
- (2) Use ``xrun`` to invoke Xcelium. You can use `xrun` with `-gui` switch to invoke the graphical interface.